



Cyberscope

A *TAC Security* Company

Audit Report

GUARD HAWK INVEST

June 2026

Files GHInvestCampaign.sol, GHInvestFactory.sol,
GHInvestGovernanceToken.sol ,GHInvestInterfaces.sol, GHInvestNFT.sol,
GHInvestProxy.sol, SteppedVoting.sol

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	6
GHInvestProxy	6
GHInvestFactory	6
GHInvestCampaign	7
GHInvestNFT	7
GHInvestGovernanceToken	8
SteppedVoting	8
Findings Breakdown	10
Diagnostics	11
AME - Address Manipulation Exploit	12
Description	12
Recommendation	12
BER - Broken Execution Retry	13
Description	13
Recommendation	14
BT - Burns Tokens	15
Description	15
Recommendation	15
CCR - Contract Centralization Risk	17
Description	17
Recommendation	17
DPI - Decimals Precision Inconsistency	18
Description	18
Recommendation	18
MT - Mints Tokens	20
Description	20
Recommendation	20
PRB - Possible Refund Block	22
Description	22
Recommendation	23
PIFR - Potential Initialization Front Running	24
Description	24
Recommendation	24
PSC - Proxy Selector Collision	25

Description	25
Recommendation	25
L02 - State Variables could be Declared Constant	26
Description	26
Recommendation	26
L04 - Conformance to Solidity Naming Conventions	27
Description	27
Recommendation	28
L15 - Local Scope Variable Shadowing	29
Description	29
Recommendation	29
L17 - Usage of Solidity Assembly	30
Description	30
Recommendation	30
Function Analysis	31
Summary	45
Disclaimer	46
About Cyberscope	47

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	11 Nov 2025 https://github.com/cyberscope-io/audits/blob/main/ghi/v1/audit.pdf
Corrected Phase 2	19 Mar 2026 https://github.com/cyberscope-io/audits/blob/main/ghi/v2/audit.pdf
Corrected Phase 3	20 May 2026 https://github.com/cyberscope-io/audits/blob/main/ghi/v3/audit.pdf
Corrected Phase 4	02 Jun 2026

Source Files

Filename	SHA256
SteppedVoting.sol	cd52f6ba9c1ab91ef4615e7535b397397b 242d70121b9a9a3d5527154775199f
GHInvestProxy.sol	19a2912e23a388eb9f19719daee90e79cb 59f6e6ca876cfa462368eda3886983
GHInvestNFT.sol	10cdbb017f7bad19b815f924fada825b27c 6e9a3bf4819b2e5c172de8b6903b1
GHInvestGovernanceToken.sol	5d4fb5f122e6e9815b71d764db0d3f39498 9c719c86dec25e76ca5fad5389a7e

GHInvestFactory.sol	4090b8ba2f4f2c04de816c28816d197f0c7 05480db964b6d5e0538bd9444dffc
GHInvestCampaign.sol	997d9ad4fbd366f20677d9a1945a024bab 15d750afa95dee2ae1d24342a4bc69

Overview

The GHInvest protocol is a milestone-based crowdfunding platform that lets entrepreneurs launch campaigns funded by stablecoin investments and governed on-chain by their investors. Each campaign is deployed as an upgradeable proxy with its own governance token and investment NFTs, and milestone-by-milestone fund releases are gated by community votes through a multi-cycle voting contract.

GHInvestProxy

A minimal EIP-1967 proxy used to deploy every campaign, governance token, and voting contract. It stores the current implementation address in the standard EIP-1967 slot and the deployer factory address in a custom slot derived from

`keccak256("ghinvest.proxy.factory")` to avoid collisions. The fallback function forwards all unmatched calls to the implementation via `delegatecall`, while the proxy itself exposes `factory()`, `implementation()`, and a factory-gated `updateImplementation(address)` administrative function. Implementation addresses must be contracts (non-zero code length), and the proxy emits events on both implementation and factory updates.

GHInvestFactory

The central hub of the platform. It deploys new campaigns and their associated governance tokens through `GHInvestProxy` instances, registers them in on-chain mappings, and tracks every campaign's entrepreneur and NFT contract. The factory owns platform-wide configuration: the campaign / token / voting implementations used for new deployments, the global investment NFT contract, the fee collector address, and the timing parameters that propagate into every new campaign (reflection period, claim grace period, and standard / fast vote intervals). All time parameters have factory-enforced minimum and maximum bounds. The factory is also the only address allowed to mint and burn the investment NFTs of a campaign, and exposes `createVotingContract` / `createFastVotingContract` so deployed campaigns can spawn `SteppedVoting` proxies for governance actions. Owner-driven upgrades to a deployed campaign's implementation are proposed through `updateDeployedCampaignImplementation`, which forwards into the campaign and triggers a community vote rather than swapping the

implementation directly. The contract is itself an `Initializable` upgradeable contract and disables initializers in its constructor.

GHInvestCampaign

`GHInvestCampaign_new.sol` is the per-campaign logic contract, deployed once and reused by every campaign proxy. It is `UUPSUpgradeable` and `ReentrancyGuardUpgradeable`, and is initialized by the factory with the campaign's hard parameters: soft and hard caps, min and max per-investor amounts, start and end dates, milestone descriptions / strategic reasons / due dates / percentages, stablecoin address, governance token address, and timing parameters. The contract drives the full campaign lifecycle: `invest` accepts stablecoin contributions, transfers governance tokens to the investor, mints an investment NFT, and enforces a per-investor reflection period during which `withdrawInvestment` allows full refund of an individual investment. `claim` releases the first milestone once the soft cap is met and the reflection period has elapsed, and is itself gated by a configurable claim grace period; subsequent milestones are released through `releaseFunds` once their due date arrives, subject to any pending milestone vote. The contract supports campaign blocking and resuming, private campaigns with invited-wallet lists, refunds for failed or cancelled campaigns, and a family of governance actions — extension of the end date, milestone-specific votes, pending-milestone delays, and implementation upgrades — each of which spawns a `SteppedVoting` instance via the factory and accepts the result through a corresponding callback. Cancellation is terminal: once the campaign is in the `Cancelled` (or `CancelledByAuthority`) state, every fund-moving and lifecycle action is disabled and only the refund path remains available to investors. Fee parameters (`investmentFeeBP` , `collectionFeeBP`) are hard-coded to zero in the current build pending regulatory clarification.

GHInvestNFT

A non-transferable ERC-721 collection representing individual investments. Every NFT records the campaign address, invested amount, investment timestamp, and per-investment terms text. The contract is deployed once and shared across all campaigns; the factory is the only minter and burner, and the campaign-to-NFT relationship is recorded in the factory at deployment time so mint / burn calls are routed against the correct campaign. Transfers between non-zero addresses are explicitly disabled in the `_update`

hook, leaving mint (from the zero address) and burn (to the zero address) as the only allowed token movements. The contract exposes a consolidated `getInvestmentDetails` view and supports ERC-165 introspection for the `IGHInvestNFT` interface.

GHInvestGovernanceToken

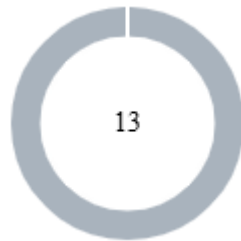
A campaign-specific ERC-20 voting token deployed per campaign by the factory. The token inherits OpenZeppelin's `ERC20Votes` to provide block-snapshotted voting power that cannot be inflated after a vote has started, and uses `ERC20Permit` for gasless approvals. The token's owner is the campaign contract itself, so only the campaign can mint, burn, or `reclaimFrom` an investor — the latter burns from the investor and mints the same amount back to the campaign without requiring approval, allowing the campaign to reverse a transfer during a withdrawal. Direct peer-to-peer transfers between non-zero addresses (other than transfers from the campaign itself) are blocked in `_beforeTokenTransfer`, so voting power remains anchored to the original investors. New recipients are automatically self-delegated in `_afterTokenTransfer` so voting power is active immediately on receipt. The token's decimals are inherited from the underlying stablecoin at deployment time and exposed through a `decimals()` override.

SteppedVoting

A multi-cycle voting contract used by `GHInvestCampaign` to authorize governance actions. Each vote can run for up to three cycles with progressively easier passing rules: the first cycle requires more than 50% participation with a simple majority; the second requires more than 30% participation with a 66% qualified majority; the third has no quorum requirement and uses a simple majority. The constructor-equivalent `initialize` snapshots the total supply of the governance token (excluding the campaign's own balance of unsold tokens) and the previous block number as the basis for voting power, sets the YES / NO calldata that will be executed against the target campaign, and starts the first cycle. Voters cast votes through `vote(bool)`, which reads weight from `getPastVotes` at the snapshot block, and `evaluateCurrentCycle` is permissionless and progresses the state once a cycle's end time is reached. On a valid cycle the vote transitions to `Finished`, the appropriate YES or NO calldata is dispatched to the target via `call`, and the success flag of that call is stored so a later retry can reattempt execution if the target reverted. Fast-vote instances skip directly to

cycle 2 with no wait period and are used for implementation upgrades through `createFastVotingContract` .

Findings Breakdown



● Critical	0
● Medium	0
● Minor / Informative	13

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	0	0	0
● Minor / Informative	13	0	0	0

Diagnostics

Severity	Code	Description	Status
●	AME	Address Manipulation Exploit	Unresolved
●	BER	Broken Execution Retry	Unresolved
●	BT	Burns Tokens	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	DPI	Decimals Precision Inconsistency	Unresolved
●	MT	Mints Tokens	Unresolved
●	PRB	Possible Refund Block	Unresolved
●	PIFR	Potential Initialization Front Running	Unresolved
●	PSC	Proxy Selector Collision	Unresolved
●	L02	State Variables could be Declared Constant	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L15	Local Scope Variable Shadowing	Unresolved
●	L17	Usage of Solidity Assembly	Unresolved

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	GHInvestFactory.sol#L661
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```
address _stablecoin
...
address campaignAddress = _deployCampaignAndToken(**params**,
_stablecoin);
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

BER - Broken Execution Retry

Criticality	Minor / Informative
Location	SteppedVoting.sol#L129,230
Status	Unresolved

Description

Before the retry branch is reachable, the cycle evaluation flow sets `state = VoteState.Finished` and only then attempts `_finalizeAndExecute`. The function `evaluateCurrentCycle` is gated by the `onlyActive` modifier, which requires `state == VoteState.Active`. A subsequent call to `evaluateCurrentCycle` therefore reverts in the modifier and never reaches the retry branch.

The most consequential case occurs in `finalizeImplementationUpdate`: if the new implementation passes the factory's ERC-165 interface check but is not properly UUPS-proxiable (for example a missing or mismatched `proxiableUUID`, which `_verifyContract` does not validate), `upgradeTo` reverts. All preceding state changes in `finalizeImplementationUpdate` are also reverted, including the clearing of `pendingImplementationUpdate`. As a result, the campaign's `pendingImplementationUpdate` remains pointing at the bad implementation, `initiateImplementationUpdate` rejects any new proposal, and the upgrade pathway is permanently locked for that campaign with no on-chain recovery path.

```
modifier onlyActive() {
    require(state == VoteState.Active, "No active voting");
    _;
}

function evaluateCurrentCycle() external onlyActive {
    VoteCycle storage c = _cycles[currentCycle];
    require(block.timestamp >= c.endTime, "Cycle not ended");
    require(!c.finished || (c.valid && !executed), "Cycle already
evaluated");
    if( c.finished ) //If finished, execution failed, retry again
    {
        _finalizeAndExecute();
    }
    else {
        ...
        if (validCycle) {
            state = VoteState.Finished;
            lastCycleEnd = block.timestamp;
            _finalizeAndExecute();
        }
        ...
    }
}
```

Recommendation

The team is advised to create a dedicated external function such as `retryExecution()` that requires `state == VoteState.Finished && _cycles[currentCycle].valid && !executed`.

BT - Burns Tokens

Criticality	Minor / Informative
Location	GHInvestGovernanceToken.sol#L112,142 GHInvestNFT.sol#L118
Status	Unresolved

Description

The contract owner has the authority to burn tokens from a specific address. The owner may take advantage of it by calling the `burn` function. As a result, the targeted address will lose the corresponding tokens.

```
function burn(address from, uint256 amount) external onlyOwner {
    _burn(from, amount);
}
...
function reclaimFrom(address from, uint256 amount) external onlyOwner
{
    _burn(from, amount);
    _mint(campaign, amount);
}
```

Additionally, the factory contract is able to burn NFT tokens.

```
function burn(uint256 tokenId) external onlyFactory {
    ...
    _burn(tokenId);
    ...
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	GHInvestFactory.sol GHInvestCampaign.sol SteppedVoting.sol GHInvestNFT.sol GHInvestGovernanceToken.sol
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

As an example the entrepreneur is able to cancel an already finalized campaign that does not meet all milestones. As a result the entrepreneur profits by collecting first milestone funds without delivering subsequent milestones.

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

DPI - Decimals Precision Inconsistency

Criticality	Minor / Informative
Location	GHInvestFactory.sol#L1095
Status	Unresolved

Description

The decimals field of a contract's ERC20 token can be used to specify the number of decimal places that the token uses. For example, if decimals are set to `8`, it means that the smallest unit of the token is `0.00000001`, and if decimals are set to `18`, it means that the smallest unit of the token is `0.00000000000000000001`.

However, there is an inconsistency in the way that the decimals field is handled in some ERC20 contracts. The ERC20 specification does not specify how the decimals field should be implemented, and as a result, some contracts use different precision numbers.

This inconsistency can cause problems when interacting with these contracts, as it is not always clear how the decimals field should be interpreted. In this case, the contract expects the decimals field of the `ghgoToken` to be equal to the decimals of the stable coin used for investing but if the stable coin used does not have the decimals function the `stablecoinDecimals` is automatically set to 18 while this might not be the actual token decimals.

```
uint8 stablecoinDecimals;
try IERC20withDecimals(params.stablecoin).decimals() returns (uint8
dec) {
    stablecoinDecimals = dec;
} catch {
    stablecoinDecimals = 18; // if decimal() not defined, use 18
    decimals.
}
```

Recommendation

To avoid these issues, it is important to carefully review the implementation of the decimals field of the underlying tokens. The team is advised to normalize each decimal to one single

source of truth. A recommended way is to scale all the decimals to the greatest token's decimal. Hence, the contract will not lose precision in the calculations.

The following example depicts 3 tokens with different decimals precision.

ERC20	Decimals
Token 1	6
Token 2	9
Token 3	18

All the decimals could be normalized to 18 since it represents the ERC20 token with the greatest digits.

MT - Mints Tokens

Criticality	Minor / Informative
Location	GHInvestGovernanceToken.sol#L101,143 GHInvestNFT.sol#L96
Status	Unresolved

Description

The contract owner has the authority to mint tokens. The owner may take advantage of it by calling the `mint` or `reclaimFrom` function. As a result, the contract tokens will be highly inflated.

```
function mint(address to, uint256 amount) external onlyOwner {
    require(!isBurned, "GHGO: All tokens have been burned");
    _mint(to, amount);
}
...
function reclaimFrom(address from, uint256 amount) external onlyOwner
{
    _burn(from, amount);
    _mint(campaign, amount);
}
```

Additionally, the factory contract is able to mint NFT tokens.

```
function mint(address _to, address _campaignAddr, uint256 _amount,
string memory _terms) external onlyFactory returns (uint256) {
    ...
    _mint(_to, tokenId);
    ...
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

PRB - Possible Refund Block

Criticality	Minor / Informative
Location	GHInvestCampaign.sol#L481,1141
Status	Unresolved

Description

The guard on the `_refund` function blocks the refund whenever an extension is pending: `require(!extensionRequested || extensionVotedFinished, "Campaign: Extension voting in process")`. This guard is unconditional and is evaluated before the state-based requirements that follow it. The `cancelCampaign` is not blocked during an extension vote.

The combination allows the following sequence: a vote-path extension is requested (so `extensionRequested = true` and `extensionVotedFinished = false`); the entrepreneur then calls `cancelCampaign` and the campaign transitions to the terminal `Cancelled` state; an investor attempts to refund and reverts, even though the campaign is already cancelled. The investor must wait for the extension vote to complete (up to three voting cycles) before refunds become available again, which can be substantially longer than the natural refund window of a cancelled campaign.

```
function cancelCampaign() external onlyEntrepreneur {
    require(state != CampaignState.Claimed, "Campaign: Already
claimed");
    require(pendingImplementationUpdate == address(0), "Campaign:
Upgrade vote in progress");
    state = CampaignState.Cancelled;
    emit CampaignCancelled();
}

function _refund(address _wallet) private {
    bool claimPeriodExpired = !claimed && block.timestamp > endDate +
claimGracePeriod;
    require(!extensionRequested || extensionVotedFinished, "Campaign:
Extension voting in process");
    require(block.timestamp > endDate || state ==
CampaignState.Cancelled || state ==
CampaignState.CancelledByAuthority, "Campaign: Not in a refundable
state");
    ...
}
```

Recommendation

The team should ensure that a cancelled campaign always allows refunds regardless of any pending extension vote. This can be expressed by bypassing the guard when the campaign is in `Cancelled` or `CancelledByAuthority`.

PIFR - Potential Initialization Front Running

Criticality	Minor / Informative
Location	GHInvestFactory.sol#L221
Status	Unresolved

Description

The contract lacks proper access control during its initialization phase, making it vulnerable to front-running attacks. An unauthorized third party could invoke the initialization process before the intended deployer, gaining control over administrative roles and critical system functions. This could lead to asset mismanagement, unauthorized fund withdrawals, or operational disruptions. Since the initializer modifier only prevents re-execution but does not restrict access, the contract is exposed to potential takeovers.

```
function initialize(address initialOwner) external initializer
```

Recommendation

To mitigate this risk, access to the initialization process should be restricted to authorized team members. This can be enforced by validating the caller against a predefined deployer address or implementing a secure deployment process that prevents external entities from intervening. Additionally, using a deployment script that finalizes initialization in the same transaction as contract creation can eliminate front-running opportunities.

PSC - Proxy Selector Collision

Criticality	Minor / Informative
Location	GHInvestProxy.sol#L64
Status	Unresolved

Description

`GHInvestProxy` reserves the external selectors for `factory()`, `implementation()`, and `updateImplementation(address)` for itself, serving them from its own storage slots before any `delegatecall` to the implementation. Function selectors are the first four bytes of the keccak256 hash of the signature, so a collision is not limited to identically-named functions: any current or future implementation function whose signature happens to hash to the same four bytes — whether it shares the proxy function's name or has a completely unrelated one — will be silently shadowed by the proxy, with no warning at compile time. Any off-chain consumer reading `factory()` from a campaign address also gets the proxy's view rather than the implementation's, so if a future implementation introduces logic that writes to the campaign's `factory` slot, the external getter will silently disagree with the new internal state.

```
function factory() public view returns (address) {  
    return _getFactory();  
}
```

Recommendation

The team should consider isolating the proxy's external surface from the selectors used by current and future implementations, so that proxy-level views and admin functions cannot shadow implementation logic. Established proxy patterns such as OpenZeppelin's Transparent Proxy address this concern by separating proxy administration from the implementation's public interface, and a similar separation could be adopted here to remove the latent risk of selector collisions over time.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	GHInvestFactory.sol#L94
Status	Unresolved

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

```
uint8 public nftTransferPolicy
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	SteppedVoting.sol#L150,151,152,153,154,155,156,203,368,382 GHInvestProxy.sol#L53 GHInvestNFT.sol#L72,96 GHInvestGovernanceToken.sol#L49 GHInvestFactory.sol#L434,472,499,653,654,655,656,657,658,659,660,661,662,663,664,665,666,923,932,967,1003 GHInvestCampaign.sol#L224,225,226,227,228,332,581
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
address _target
bytes memory _callDataIfYes
bytes memory _callDataIfNo
uint256 _votingDuration
uint256 _waitPeriod
address _governanceToken
uint8 _initialCycle
bool _support
address _user
uint8 _cycle
address _newImplementation
address _factoryAddress
address _campaignAddr
string memory _terms

...
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

L15 - Local Scope Variable Shadowing

Criticality	Minor / Informative
Location	GHInvestInterfaces.sol#L125
Status	Unresolved

Description

Local scope variable shadowing occurs when a local variable with the same name as a variable in an outer scope is declared within a function or code block. When this happens, the local variable "shadows" the outer variable, meaning that it takes precedence over the outer variable within the scope in which it is declared.

```
bool isPrivate
```

Recommendation

It's important to be aware of shadowing when working with local variables, as it can lead to confusion and unintended consequences if not used correctly. It's generally a good idea to choose unique names for local variables to avoid shadowing outer variables and causing confusion.

L17 - Usage of Solidity Assembly

Criticality	Minor / Informative
Location	GHInvestProxy.sol#L35,70,78,86,93
Status	Unresolved

Description

Using assembly can be useful for optimizing code, but it can also be error-prone. It's important to carefully test and debug assembly code to ensure that it is correct and does not contain any errors.

Some common types of errors that can occur when using assembly in Solidity include Syntax, Type, Out-of-bounds, Stack, and Revert.

```
assembly {  
    // Copy msg.data  
    calldatacopy(0, 0, calldatasize())  
  
    // Forward call to implementation  
    let result := delegatecall(gas(), _impl, 0,  
    calldatasize(), 0, 0)  
    ...  
    returndatacopy(0, 0, returndatasize())  
  
    // Return result  
    switch result  
    case 0 { revert(0, returndatasize()) }  
    default { return(0, returndatasize()) }  
}  
  
...
```

Recommendation

It is recommended to use assembly sparingly and only when necessary, as it can be difficult to read and understand compared to Solidity code.

Function Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
SteppedVoting	Implementation	Initializable, ISteppedVoting, ERC165Upgradable		
	initialize	Public	✓	initializer
		Public	✓	-
	vote	External	✓	onlyActive
	evaluateCurrentCycle	External	✓	onlyActive
	advanceToNextCycle	External	✓	-
	_advanceToNextCycle	Internal	✓	
	_startCycle	Internal	✓	
	_finalizeAndExecute	Internal	✓	
	getVoteState	External		-
	hasVoted	External		-
	getCycleVotes	External		-
	supportsInterface	Public		-
GHIInvestProxy	Implementation			
		Public	✓	-
		External	✓	-

Contract	Type	Bases		
	updateImplementation	External	✓	-
	implementation	Public		-
	factory	Public		-
	_getImplementation	Private		
	_setImplementation	Private	✓	
	_getFactory	Private		
	_setFactory	Private	✓	
GHInvestNFT	Implementation	ERC721Enumerable, Ownable, IGHInvestNFT		
		Public	✓	ERC721 Ownable
	setFactory	External	✓	onlyOwner
	supportsInterface	Public		-
	mint	External	✓	onlyFactory
	burn	External	✓	onlyFactory
	getInvestmentDetails	External		-
	balanceOf	Public		-
	ownerOf	Public		-
	tokenOfOwnerByIndex	Public		-
	totalSupply	Public		-
	_update	Internal	✓	

Contract	Type	Bases		
IGHInvestCamp aign	Interface	IERC165		
	factory	External		-
	entrepreneur	External		-
	softCap	External		-
	hardCap	External		-
	minAmount	External		-
	maxAmount	External		-
	startDate	External		-
	endDate	External		-
	stablecoin	External		-
	raisedAmount	External		-
	claimed	External		-
	currentState	External		-
	name	External		-
	isPrivate	External		-
	investedAmount	External		-
	investorList	External		-
	investorListLength	External		-
	getInvestorList	External		-
	getFrozenAmount	External		-
	getIndividualInvestments	External		-

Contract	Type	Bases		
	isInvestmentFrozen	External		-
	getMyIndividualInvestments	External		-
	milestones	External		-
	milestonesLength	External		-
	getMilestones	External		-
	getBalance	External		-
	isCampaignSuccessful	External		-
	isHardCapReached	External		-
	ghgoToken	External		-
	getTermsAndConditions	External		-
	getCurrentPendingMilestoneExtension	External		-
	getAllPendingMilestoneExtensions	External		-
	getExtensionStatus	External		-
	getImplementationUpdateRequest	External		-
	pendingImplementationUpdate	External		-
	initialize	External	✓	-
	invest	External	✓	-
	claim	External	✓	-
	withdrawInvestment	External	✓	-
	refund	External	✓	-
	refundTo	External	✓	-
	setPrivate	External	✓	-

Contract	Type	Bases		
	addInvitedWallet	External	✓	-
	removeInvitedWallet	External	✓	-
	addInvitedWallets	External	✓	-
	requestExtension	External	✓	-
	extendEndDate	External	✓	-
	rejectExtension	External	✓	-
	requestMilestoneVote	External	✓	-
	finalizeMilestoneVote	External	✓	-
	responseMilestoneVote	External	✓	-
	requestPendingMilestoneDelay	External	✓	-
	finalizeRequestPendingMilestoneDelayVote	External	✓	-
	cancelCampaign	External	✓	-
	blockCampaign	External	✓	-
	resumeCampaign	External	✓	-
	releaseFunds	External	✓	-
	initiateImplementationUpdate	External	✓	-
	finalizeImplementationUpdate	External	✓	-
IGHInvestFactory	Interface	IERC165		
	campaignImplementation	External		-
	deployedCampaigns	External		-
	validatedWallets	External		-

Contract	Type	Bases		
	isCampaign	External		-
	campaignToEntrepreneur	External		-
	entrepreneurToCampaigns	External		-
	feeCollector	External		-
	voteCallInterval	External		-
	voteCallIntervalFast	External		-
	voteIntervalBetweenCalls	External		-
	reflectionPeriod	External		-
	deployCampaign	External	✓	-
	validateWallet	External	✓	-
	invalidateWallet	External	✓	-
	updateCampaignImplementation	External	✓	-
	createVotingContract	External	✓	-
	createFastVotingContract	External	✓	-
	mintNft	External	✓	-
	burnNft	External	✓	-
	getAllCampaigns	External		-
	getCampaignsByEntrepreneur	External		-
	getCampaignsByState	External		-
	getCampaignMetadata	External		-
	isWalletValidated	External		-
	isDeployedCampaign	External		-

Contract	Type	Bases		
	getCampaignImplementation	External		-
	getCampaignStats	External		-
	getDetailedCampaignStats	External		-
IGHInvestNFT	Interface	IERC165, IERC721Enumerable		
	mint	External	✓	-
	burn	External	✓	-
	ownerOf	External		-
	balanceOf	External		-
	tokenOfOwnerByIndex	External		-
	totalSupply	External		-
	setFactory	External	✓	-
	getInvestmentDetails	External		-
IGHInvestGovernanceToken	Interface	IERC165Upgradeable		
	initialize	External	✓	-
	decimals	External		-
	mint	External	✓	-
	burn	External	✓	-
	reclaimFrom	External	✓	-
	balanceOf	External		-

Contract	Type	Bases		
	totalSupply	External		-
	transfer	External	✓	-
	transferFrom	External	✓	-
	approve	External	✓	-
	allowance	External		-
	getPastVotes	External		-
ISteppedVoting	Interface	IERC165Up gradeable		
	initialize	External	✓	-
GHInvestGovern anceToken	Implementation	Initializable, ERC20Upgr adeable, ERC20Permi tUpgradeabl e, ERC20Votes Upgradeabl e, OwnableUp gradeable, IGHInvestGo vernanceTok en, ERC165Upg radeable		
	initialize	Public	✓	initializer
		Public	✓	-
	decimals	Public		-
	supportsInterface	Public		-
	mint	External	✓	onlyOwner
	burn	External	✓	onlyOwner

Contract	Type	Bases		
	reclaimFrom	External	✓	onlyOwner
	_beforeTokenTransfer	Internal	✓	
	_afterTokenTransfer	Internal	✓	
	allowance	Public		-
	balanceOf	Public		-
	totalSupply	Public		-
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	approve	Public	✓	-
	getPastVotes	Public		-
	delegate	Public	✓	-
	_mint	Internal	✓	
	_burn	Internal	✓	
IGHInvestProxy	Interface			
	updateImplementation	External	✓	-
IERC20withDecimals	Interface			
	decimals	External		-
GHInvestFactory	Implementation	Initializable, OwnableUp gradeable, IGHInvestFa ctory		

Contract	Type	Bases		
	_verifyContract	Internal		
	initialize	External	✓	initializer
		Public	✓	-
	setSystemAddress	External	✓	onlyOwner
	updateCampaignImplementation	External	✓	onlyOwner
	updateTokenImplementation	External	✓	onlyOwner
	updateVotingImplementation	External	✓	onlyOwner
	updateDeployedCampaignImplementation	External	✓	onlyOwner
	setNFTContract	External	✓	onlyOwner
	setFeeCollector	External	✓	onlyOwner
	setReflectionPeriod	External	✓	onlyOwner
	setVoteCallInterval	External	✓	onlyOwner
	setVoteCallIntervalFast	External	✓	onlyOwner
	setVoteIntervalBetweenCalls	External	✓	onlyOwner
	setClaimGracePeriod	External	✓	onlyOwner
	mintNft	External	✓	onlyCampaign
	burnNft	External	✓	onlyCampaign
	deployCampaign	External	✓	onlyOwnerOrSystem
	createVotingContract	External	✓	onlyCampaign
	createFastVotingContract	External	✓	onlyCampaign
	validateWallet	External	✓	onlyOwnerOrSystem
	invalidateWallet	External	✓	onlyOwnerOrSystem

Contract	Type	Bases		
	getAllCampaigns	External		-
	getCampaignsByEntrepreneur	External		-
	getCampaignsByState	External		-
	getCampaignMetadata	External		-
	isWalletValidated	External		-
	isDeployedCampaign	External		-
	getCampaignImplementation	External		-
	getCampaignStats	External		-
	getDetailedCampaignStats	External		-
	supportsInterface	Public		-
	_deployCampaignAndToken	Private	✓	
GHIInvestCampaign	Implementation	Initializable, UUPSUpgradeable, ERC165Upgradeable, ReentrancyGuardUpgradeable, IGHInvestCampaign		
		Public	✓	-
	initialize	External	✓	initializer
	invest	External	✓	onlyNotCancelled
	withdrawInvestment	External	✓	onlyNotCancelled
	claim	External	✓	onlyEntrepreneur

Contract	Type	Bases		
				onlyNotCancel ed
	refund	External	✓	-
	refundTo	External	✓	onlyEntreprene ur
	releaseFunds	External	✓	onlyEntreprene ur onlyNotCancel ed
	cancelCampaign	External	✓	onlyEntreprene ur
	blockCampaign	External	✓	onlyEntreprene ur
	resumeCampaign	External	✓	onlyEntreprene ur
	setPrivate	External	✓	onlyEntreprene ur onlyActive
	addInvitedWallet	External	✓	onlyEntreprene ur onlyActive
	removeInvitedWallet	External	✓	onlyEntreprene ur onlyActive
	addInvitedWallets	External	✓	onlyEntreprene ur onlyActive
	requestExtension	External	✓	onlyEntreprene ur onlyActive
	extendEndDate	External	✓	-
	rejectExtension	External	✓	-
	_applyDateExtension	Internal	✓	
	requestMilestoneVote	External	✓	-
	responseMilestoneVote	External	✓	onlyEntreprene ur
	_respondMilestoneVote	Internal	✓	
	callMilestoneVote	External	✓	-

Contract	Type	Bases		
	finalizeMilestoneVote	External	✓	-
	requestPendingMilestoneDelay	External	✓	onlyEntrepreneur
	finalizeRequestPendingMilestoneDelayVote	External	✓	-
	initiateImplementationUpdate	External	✓	onlyFactory
	_authorizeUpgrade	Internal	✓	
	finalizeImplementationUpdate	External	✓	-
	currentState	External		-
	getInvestorList	External		-
	getMilestones	External		-
	getBalance	External		-
	isCampaignSuccessful	External		-
	isHardCapReached	External		-
	getFrozenAmount	External		-
	getIndividualInvestments	External		-
	getMyIndividualInvestments	External		-
	isInvestmentFrozen	External		-
	getExtensionStatus	External		-
	getImplementationUpdateRequest	External		-
	getCurrentPendingMilestoneExtension	External		-
	getAllPendingMilestoneExtensions	External		-
	getTermsAndConditions	External		-

Contract	Type	Bases		
	supportsInterface	Public		-
	investorListLength	External		-
	milestonesLength	External		-
	milestones	External		-
	_invest	Private	✓	nonReentrant onlyInvestor
	_refund	Private	✓	
	_getCurrentMilestone	Private		
	_createVoting	Internal	✓	
	_requireNoActiveVote	Internal		

Summary

GUARD HAWK INVEST contract implements a financial, governance, exchange and utility mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io